

**A Fault Tolerant I2C Master Controller for CubeSat
Applications**

A Minor Project work submitted in partial fulfillment of the requirement for the
award of the degree of

BACHELOR OF TECHNOLOGY
in
ELECTRONICS & COMMUNICATION ENGINEERING

by

N. Sathwik Reddy (24211A04G1)
Odugu Sruthi (24211A04G8)
P.V. Lalith Surya (24211A04G9)

Under the esteemed guidance of

Dr. K. Madhava Rao
Assistant Professor



B.V.Raju Institute of Technology
UGC- AUTONOMOUS
Department of Electronics and Communication Engineering
Vishnupur, Narsapur, Medak.(Dt)

(Affiliated to JNTU, Hyderabad)
2025-2026
B.V.Raju Institute of Technology
Vishnupur, Narsapur, Medak.(Dt) Pin:502313
(Affiliated to JNTU, Hyderabad)
Ph: 08458-222000, 222001 Fax: 08458-222002

Department of Electronics & Communication Engineering



CERTIFICATE

This is to certify that the Minor Project work entitled “**A Fault-Tolerant I2C Master Architecture for CubeSat Applications**” is being submitted by Mr. N. Sathwik Reddy, Ms. Odugu Sruthi, Mr. PV Lalith Surya in partial fulfillment of the requirement for the award of the degree of **B.Tech in Electronics & Communication Engineering**, by Jawaharlal Nehru Technological University Hyderabad is a record of bonafide work carried out by him under my guidance and supervision from December 2025 to May 2026 .

The results presented in this project have been verified and are found to be satisfactory.

INTERNAL GUIDE
Dr. K. Madhava Rao
M. Tech, Ph.d
Assistant Professor

Dr. B R Sanjeeva Reddy
B.E, M.Tech, PhD.
Professor & HOD, Dept. of ECE

CENTRE FOR VLSI DESIGN

Department of Electronics and Communication Engineering



Certificate

This is to certify that Mr. N. Sathwik Reddy, Ms. Odugu Sruthi , Mr. P.V. Lalith Surya have successfully implemented a project titled “**Design and Implementation of Clock Domain Crossing using Gray code-based Asynchronous FIFO**” in Centre of VLSI Design, Department of Electronics and Communication, B.V. Raju Institute of Technology, Narsapur, during the period from 2025-2026.

LAB COORDINATOR:

Dr. U. Gnaneshwara Chary, PhD

Assistant Professor

ACKNOWLEDGEMENTS

I would like to express our sincere gratitude to all those who have supported and guided us in the successful completion of our minor project.

First and foremost, I extend our heartfelt thanks to our beloved principal Dr. Sanjay Dubey for providing us with the opportunity to undertake this project and for creating an academic environment that encourages innovation and learning.

We are also extremely thankful to Dr. B. R. Sanjeeva Reddy, Ph. D, MIEEE, Professor & HOD, Department of ECE for their support, motivation, and for facilitating the resources required for the project.

Our deepest appreciation goes to our project guide Dr. K. Madhava Rao, Ph. D, for their constant guidance, valuable insights, and continuous encouragement throughout the project. Their expertise and mentorship played a key role in the successful completion of our work.

We also wish to thank all the faculty members and staff of the department for their cooperation and support.

This project has been a valuable and enriching experience, and we are thankful for the opportunity to work together as a team.

Regards,
N. Sathwik Reddy (24211A04G1)
Odugu Sruthi (24211A04G8)
P.V. Lalith Surya (24211A04G9)

ABSTRACT

The Inter-Integrated Circuit (I2C) protocol is heavily favored for CubeSat telemetry systems due to its low power consumption, space efficiency, and architectural simplicity. However, in the harsh radiation environment of space, standard software-driven I2C controllers are highly vulnerable to Single Event Transients (SETs) and Single Event Upsets (SEUs). High-energy cosmic particles induce temporary voltage spikes that act as false data bits, causing catastrophic bus deadlocks and critical mission failures. Because CubeSats operate under strict mass and volume constraints, traditional physical radiation shielding is unfeasible, necessitating a digital mitigation approach. This project presents the design and implementation of a custom, radiation-hardened I2C Master Controller engineered in Verilog HDL. The architecture replaces vulnerable software bit-banging with a deterministic 9-state Finite State Machine (FSM) for reliable physical bus arbitration. Furthermore, the design integrates a Digital Temporal Voting Filter—a 3-stage oversampling shift register—to autonomously identify and reject sub-cycle radiation spikes. Simulated fault-injection testing confirms the controller's ability to seamlessly trap a 15-nanosecond radiation strike and maintain data transmission without entering infinite wait states. This digital shielding methodology ensures uninterrupted telemetry communication, establishing a highly reliable hardware foundation for space-grade nanosatellite operations.

Keywords: I2C Protocol, Radiation Hardening, Fault Tolerance, Finite State Machine (FSM), Single Event Upset (SEU), Verilog HDL, CubeSat Telemetry, Temporal Voting Filter.

CONTENTS

CERTIFICATE	I&II
ACKNOWLEDGEMENTS	III
ABSTRACT	IV
CONTENTS	V
LIST OF FIGURES	VI
1. INTRODUCTION	1
1.1 Motivation	1
1.2 Objective	2
1.3 Scope	2
2. LITERATURE SURVEY	3
3. ANALYSIS & DESIGN	7
3.1 Overview of the I2C Protocol	7
3.2 Proposed System Architecture	7
3.3 Master Controller FSM Design	8
3.4 Radiation Hardening: The Temporal Voting Filter	9
3.5 Clock Division and Synchronization	10
4. IMPLEMENTATION	10
4.1 Implementation Environment	10
4.2 Clock Division and Synchronization Logic	10
4.3 Radiation-Hardened Temporal Voting Filter	11
4.4 Finite State Machine (FSM) Implementation	12
4.5 Bidirectional Bus Control (Tri-State Buffers)	13
5. RESULTS	14
5.1 Simulation Setup and Environment	14
5.2 Test Case 1: Standard Write Operation	14
5.3 Test Case 2: Standard Read Operation	15
5.4 Test Case 3: Radiation Fault Injection and Mitigation	15
6. CONCLUSION AND FUTURE WORK	17
6.1 Conclusion	17
6.2 Future Scope	17
REFERENCES & APPENDIX	18

LIST OF FIGURES

Figure 3.1 Standard I2C Master-Slave Bus Architecture	7
Figure 3.2 Proposed System Level Architecture of the Master Controller	8
Figure 3.3 9-State Finite State Machine (FSM) Logic Flow	9
Figure 3.4 3-Stage Temporal Voting Filter Mechanism	9
Figure 5.1: I2C Write Operation	14
Figure 5.2: I2C Read Operation Capturing Slave Data	15
Figure 5.3: Fault Injection Simulation Demonstration	16

1. INTRODUCTION

1.1 Motivation

Space technology represents the fastest-growing sector within modern deep-tech, a surge driven almost entirely by the proliferation of nanosatellites. In 2025 alone, nearly 400 CubeSats were launched into orbit, representing a staggering 38% growth rate compared to the previous year. However, this rapid expansion masks a hidden operational crisis: most commercial off-the-shelf (COTS) CubeSats fly blind in harsh environments because they are not properly radiation-hardened. Over the last decade, more than fifteen high-profile small satellite missions have completely failed directly due to internal communication breakdowns.

The core of this problem lies in the Inter-Integrated Circuit (I2C) protocol. Almost every CubeSat utilizes I2C to connect the On-Board Computer (OBC) to critical telemetry sensors like gyroscopes, magnetometers, and temperature gauges because it is remarkably simple, space-efficient, and cost-effective. Unfortunately, standard I2C architecture is fundamentally fragile. It is highly susceptible to Single Event Upsets (SEUs) and Single Event Transients (SETs). A single high-energy radiation strike can flip a bit on the unshielded wire, acting as a false data signal. Because standard software-based controllers trust the wire implicitly, this transient noise is easily misinterpreted as a false STOP condition or an acknowledgment failure, permanently locking up the sensor bus and corrupting vital telemetry data. Since traditional physical radiation shielding—such as thick lead or aluminum casing—is impossible given a CubeSat's strict weight and volume limits, an embedded digital shielding solution is required to ensure mission survival.

1.2 Objective

- The primary objective of this project is to engineer a fault-tolerant, radiation-hardened I2C Master Controller tailored specifically for aerospace applications. The specific goals include:
- **Hardware-Driven Determinism:** To design a custom 9-state Finite State Machine (FSM) using Verilog HDL that handles strict physical bus arbitration, replacing vulnerable software-level protocol management.
- **Digital Noise Filtering:** To develop and integrate a Digital Temporal Voting Filter capable of oversampling the data line at high clock speeds, autonomously differentiating between legitimate sensor data and radiation-induced transient voltage spikes.
- **Seamless Fault Mitigation:** To ensure the system can absorb hardware-level electrical disturbances and complete data transactions without requiring a full system reboot or dropping the active payload.

1.3 Scope

The scope of this project encompasses the architectural design, coding, and behavioral simulation of the custom I2C Master interface. The design is implemented at the Register Transfer Level (RTL) using Verilog HDL and verified via the Xilinx Vivado Design Suite. The functional scope focuses on establishing standard Read and Write operations combined with advanced digital glitch filtering. Verification is limited to rigorous simulated fault-injection environments—specifically blasting the data lines with targeted 15-nanosecond artificial radiation strikes during critical transaction phases to prove filter efficacy. The project culminates in a verified intellectual property (IP) core that provides a scalable, software-independent digital shield for CubeSat telemetry buses, entirely circumventing the need for expensive physical analog isolation circuits.

2. Literature Survey

S.No	Year	Title & Authors	Technology / Methods	Merits / Findings	Demerits / Limitations	Applications
1	2016	<i>Survey on the implementation and reliability of CubeSat electrical bus interfaces</i> (J. Bouwmeester et al.)	In-orbit failure analysis and statistical tracking of CubeSat telemetry buses.	Identified the standard I2C protocol as a primary cause of catastrophic mission failures and bus lockups in Low Earth Orbit (LEO).	Provides only statistical analysis of failures; does not propose a specific hardware-level mitigation design.	CubeSat telemetry, satellite reliability analysis.
2	2017	<i>A CubeSat-Payload Radiation-Reliability Assurance Case</i> (Austin et al.)	System-level Single Event Latch-up (SEL) detection using load switches and external watchdog timers (WDT).	Validated that external hardware watchdogs can successfully reset microcontroller s during radiation-induced lockups (SEFI).	Relies on power-cycling the entire system, causing mission downtime and losing volatile state data.	CubeSat science payloads, COTS component shielding.
3	2020	<i>Mitigating single event upset of FPGA for the onboard bus control of satellite</i> (Zhao et al.)	Hybrid anti-radiation architecture combining Error Detection and Correction	Successfully corrected 1-bit errors and detected 2-bit errors in FPGA control registers dynamically.	TMR introduces massive area overhead (3x logic) and significantly increases FPGA power consumption.	Commercial satellite onboard computers, FPGA bus controllers.

S.No	Year	Title & Authors	Technology / Methods	Merits / Findings	Demerits / Limitations	Applications
			(EDAC) with Triple Modular Redundancy (TMR).			
4	2021	<i>Circuit-level techniques for mitigating radiation-induced degradation of commercial microelectronics in space</i> (Lofgren)	Dynamic biasing and serial bus isolation circuits for I2C and SPI running at 400kHz.	Effectively eliminated single-point failure modes on the SDA/SCL lines without requiring additional FPGA I/O pins.	Requires custom analog isolation circuits on the PCB rather than a purely digital, embeddable HDL solution.	Small-spacecraft avionics, deep-space microelectronics.
5	2023	<i>Fault Analysis and Mitigation Techniques of the I2C Bus for Nanosatellite Missions</i> (A. Albalooshi et al.)	Experimental fault injection simulating missing SCL pulses, bit-flips, and stuck-at-0 faults on the SDA line.	Proved that the inherent lack of data integrity checks in I2C makes it fatally vulnerable to missing acknowledgments.	Focuses heavily on isolating faulty nodes physically rather than filtering transient noise (glitches) at the master logic level.	Nanosatellite data handling, I2C bus stress testing.
6	2024	<i>Fault-Tolerant FPGA-Based System for Mitigating</i>	Optimization of FPGA logic cells and Finite State	Highlighted that standard, area-optimized FPGA FSMs will enter	Highly theoretical; assumes the developer has deep access to	High-reliability FPGA synthesis, mission-

S.No	Year	Title & Authors	Technology / Methods	Merits / Findings	Demerits / Limitations	Applications
		<i>SEUs in Configuration and User Bits</i> (O. T. Amer et al.)	Machines (FSMs) for high-radiation environments.	infinite wait-states when exposed to SEU-induced bit-flips.	background FPGA configuration memory (CRAM) scrubbing.	critical state logic.
7	2025	<i>Enhanced I2C Architecture With Integrated CRC for Error Detection in Multi-Slave Systems</i> (Suchitra M et al.)	Integration of Cyclic Redundancy Check (CRC) algorithms directly into the I2C data transmission protocol.	Replaces the standard, fragile ACK/NACK reliance with strict payload verification, completely preventing data corruption.	Increases transaction payload size and requires custom I2C slaves (breaks compatibility with standard COTS sensors).	Multi-slave communication, secure embedded systems.
8	2025	<i>FPGA improved I2C protocol for multiple slave device using VHDL</i> (A. Mishra et al.)	Baseline FPGA implementation of standard I2C master/slave bidirectional logic.	Provides an excellent baseline framework for optimizing FPGA chip area and clock division in noise-free environments.	Highly vulnerable to radiation; lacks glitch filtering, input sanitization, or deadlock recovery systems.	General purpose IoT, industrial sensor interfacing.
9	2025	<i>Ensuring I2C Bus Reliability in</i>	Experimental laboratory testing	Provides a comprehensive breakdown of	Mitigation primarily relies on software-level	CubeSat operations, I2C physical

S.No	Year	Title & Authors	Technology / Methods	Merits / Findings	Demerits / Limitations	Applications
		<i>Nanosatellite Operations Fault Analysis and Mitigation Strategies</i> (G. Ajitha et al.)	combined with software-level checksum strategies.	how parasitic capacitance and pull-up resistor values exacerbate I2C lockups.	checksums, which are too slow to catch sub-nanosecond radiation spikes.	layer verification.
10	2025	<i>Radiation Effects in FPGAs and SoCs</i> (P. Maillard)	Fault Aware Training and Single Event Effect (SEE) validation for modern SRAM-based FPGA fabrics.	Demonstrates that even the newest commercial FPGA logic cells are highly susceptible to transient voltage spikes.	Focused on massive Datacenter/Avionics FPGAs rather than power-constrained CubeSat controllers.	Terrestrial avionics, AMD/Xilinx space-grade FPGAs.

3. ANALYSIS AND DESIGN

3.1 Overview of the I2C Protocol

The Inter-Integrated Circuit (I2C) is a synchronous, multi-master, multi-slave, packet-switched, single-ended serial communication bus. It uses only two bidirectional open-drain lines: Serial Data Line (SDA) and Serial Clock Line (SCL), pulled up with resistors. While standard I2C is highly efficient for localized terrestrial systems, its reliance on voltage levels to determine critical protocol commands (such as START and STOP conditions) makes it highly susceptible to Single Event Transients (SETs) in high-radiation environments like Low Earth Orbit (LEO).

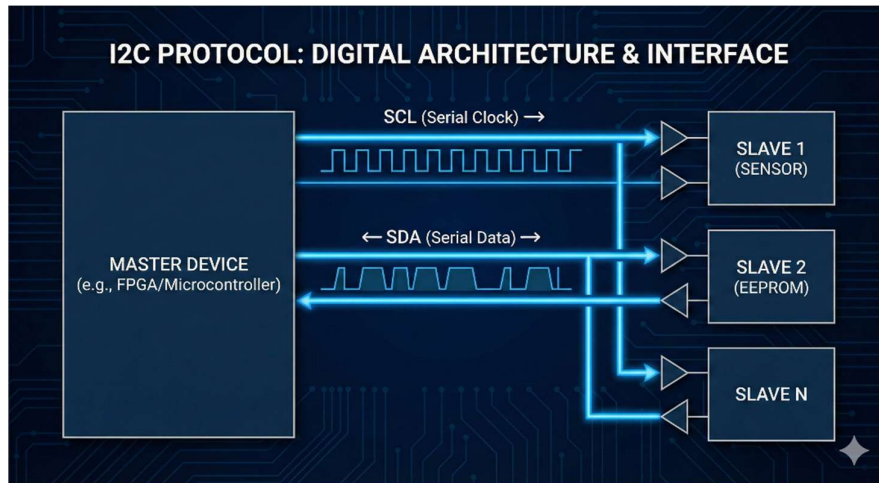


Figure 3.1: Standard I2C Master-Slave Bus Architecture

3.2 Proposed System Architecture

To overcome the vulnerabilities of software-driven I2C controllers, this project proposes a completely hardware-driven, deterministic Master Controller. The architecture is logically divided into three primary modules: the Host Interface (Capture Logic), the Master Finite State Machine (The Brain), and the Physical I/O Interface containing the digital radiation shielding. The Host Interface synchronizes the system clock and safely latches the incoming 7-bit addresses and 8-bit payloads to prevent mid-transaction data corruption.

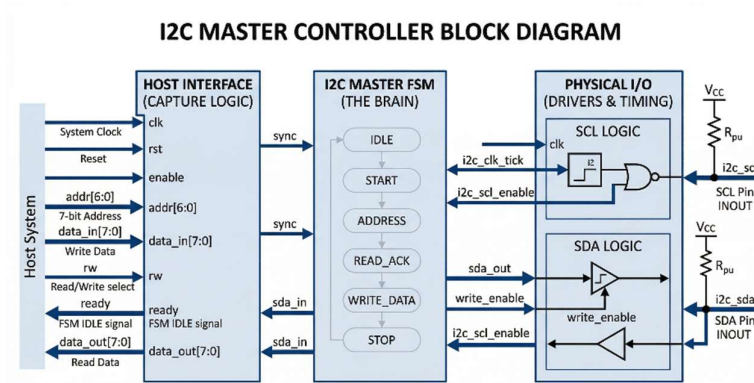


Figure 3.2: Proposed System Level Architecture of the Master Controller

3.3 Master Controller FSM Design

- The core operational logic of the Master is governed by a 9-state Mealy Finite State Machine (FSM) implemented in Verilog HDL. By relying on a strict hardware FSM rather than software interrupts, the controller eliminates delta-cycle timing violations and race conditions. The states are defined as follows:
- **IDLE:** The default resting state. The master drives the bus high (High-Z) and waits for the enable trigger.
- **START:** Generates the physical START condition by pulling SDA low while SCL remains high.
- **ADDRESS:** Serially transmits the 7-bit slave address followed by the 1-bit Read/Write (R/W) instruction over 8 clock cycles.
- **READ_ACK:** Releases the bus and samples the SDA line to check if the targeted slave pulled it low (Acknowledged).
- **WRITE_DATA / READ_DATA:** Depending on the R/W bit, the FSM either transmits 8 bits of payload data to the slave or samples 8 bits of incoming data from the slave.
- **WRITE_ACK / READ_ACK2:** Processes the acknowledgment bit for the payload data.
- **STOP:** Generates the physical STOP condition by releasing SDA high while SCL is high, cleanly terminating the transaction and returning to IDLE.

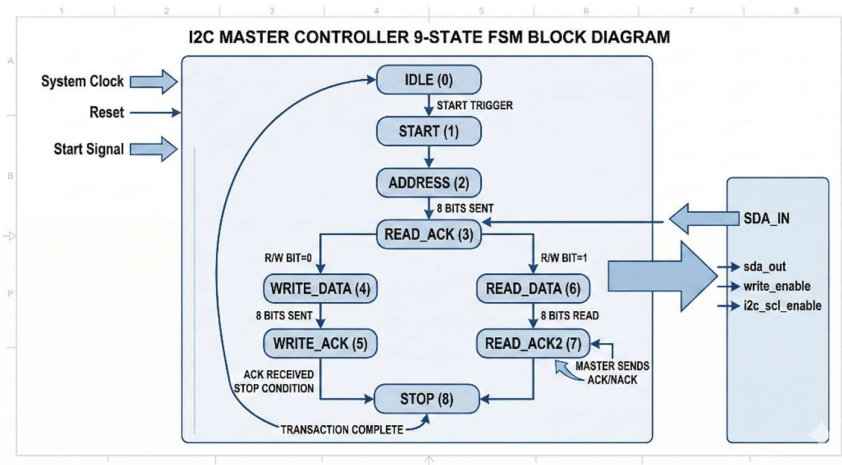


Figure 3.3: 9-State Finite State Machine (FSM) Logic Flow

3.4 Radiation Hardening: The Temporal Voting Filter

The most critical innovation in this design is the Digital Temporal Voting Filter, engineered to mitigate Single Event Transients (SETs). In a space environment, a high-energy particle striking the SDA line can create a transient voltage spike. A standard controller reading the bus at that exact moment would misinterpret the spike as a NACK or a STOP condition, aborting the mission-critical transaction. To prevent this, the proposed controller heavily oversamples the physical SDA line using the fast 50MHz system clock. The raw physical signal is fed into a 3-stage digital shift register. A strict "Majority Voting" logic algorithm is applied: the internal filtered_sda variable—which the FSM relies on—will only update its state if the shift register captures three identical, consecutive samples (representing 60 nanoseconds of stable voltage)

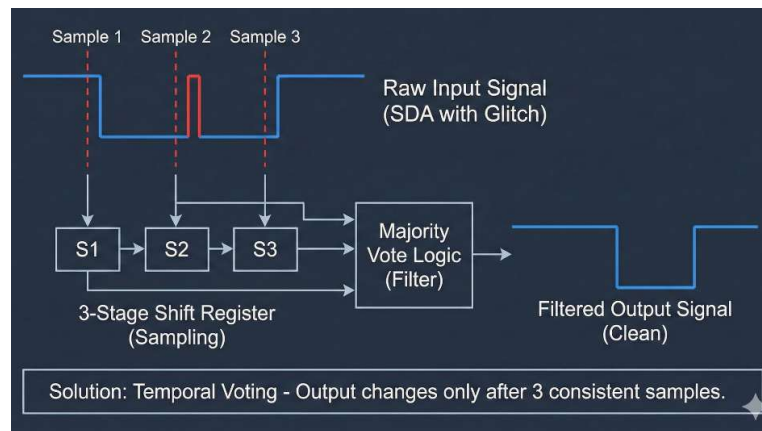


Figure 3.4: 3-Stage Temporal Voting Filter Mechanism

3.5 Clock Division and Synchronization

To accommodate the latency introduced by the digital filter, careful clock synchronization is required. The glitch filter requires 60ns to output a verified signal. Therefore, the fast 50MHz system clock is passed through a clock divider set to a ratio of 20. This generates a stable, internal I2C clock running at 2.5 MHz (400ns period). This deliberate division ensures that the FSM's setup time is significantly longer than the filter's processing latency, entirely eliminating internal read-before-write timing collisions.

4. IMPLEMENTATION

4.1 Implementation Environment

The custom I2C Master Controller was implemented at the Register Transfer Level (RTL) using Verilog Hardware Description Language (HDL), adhering to the IEEE 1364 standard. The design, synthesis, and behavioral simulation were executed utilizing the Xilinx Vivado Design Suite. The architecture was specifically coded to avoid software-level dependencies, utilizing purely sequential and combinational digital logic blocks to ensure deterministic execution in radiation-prone environments.

Note: To facilitate clear explanations of the core logic, only essential Verilog snippets are presented within this chapter. The complete, compilable source code for all modules and testbenches is provided in the Appendix.

4.2 Clock Division and Synchronization Logic

To satisfy the timing requirements of the I2C protocol and provide sufficient latency for the digital filter, a clock divider was implemented. The system utilizes a fast 50 MHz base clock (clk). By setting a local parameter DIVIDE_BY to 20, the system generates a stable 2.5 MHz internal I2C clock (i2c_clk) with a period of 400 nanoseconds.

Verilog:

```
// Clock Divider Logic
localparam DIVIDE_BY = 20;
reg [7:0] counter2 = 0;
reg i2c_clk = 1;

always @(posedge clk) begin
    if (counter2 == (DIVIDE_BY/2) - 1) begin
        i2c_clk <= ~i2c_clk;
        counter2 <= 0;
    end
    else counter2 <= counter2 + 1;
end
```

This division is critical: it guarantees that the low-phase of the I2C clock (200ns) is comfortably longer than the Temporal Voting Filter's processing time (60ns), preventing race conditions when the FSM reads the filtered data.

4.3 Radiation-Hardened Temporal Voting Filter

The digital shielding mechanism was implemented using a 3-bit shift register (sda_sr). This register operates on the fast 50 MHz system clock, taking rapid, continuous samples of the physical i2c_sda pin. The FSM is restricted from reading the physical wire directly; instead, it reads the filtered_sda register. The filter strictly requires three consecutive identical samples to update its output.

Verilog:

```
// --- GLITCH FILTER LOGIC ---
reg [2:0] sda_sr = 3'b111; // 3-bit shift register, defaults to 1 (bus idle)
reg filtered_sda = 1; // The clean signal the FSM will use

always @(posedge clk or posedge rst) begin
    if (rst == 1) begin
        sda_sr <= 3'b111;
    end
```

```

        filtered_sda <= 1'b1;
    end else begin
        // Shift the raw physical SDA pin value into the register
        sda_sr <= {sda_sr[1:0], i2c_sda};

        // Strict voting: All 3 consecutive samples must agree
        if (sda_sr == 3'b111) begin
            filtered_sda <= 1'b1;
        end else if (sda_sr == 3'b000) begin
            filtered_sda <= 1'b0;
        end
        // Any other state (e.g., 010, 110) implies a radiation transient,
        // and the filter maintains its previous stable value.
    end
end
end

```

4.4 Finite State Machine (FSM) Implementation

The core control logic is governed by a 9-state Mealy FSM. To strictly adhere to the physical laws of the I2C protocol—which dictates that data must remain stable while the clock is high—the FSM was explicitly coded to transition states and shift data exclusively on the negative edge (negedge) of the i2c_clk.

Verilog:

```

// Partial FSM Implementation (Data Shifting)
always @(negedge i2c_clk, posedge rst) begin
    if(rst == 1) begin
        state <= IDLE;
    end else begin
        case(state)
            // Example: Transmission State
            WRITE_DATA: begin
                if (counter == 0) begin
                    state <= WRITE_ACK;
                end
            end
        endcase
    end
end

```

```

        end else counter <= counter - 1;
    end

    // Example: Reception State
    READ_DATA: begin
        // Capture data from the filtered signal
        data_out[counter] <= filtered_sda;
        if (counter == 0) state <= READ_ACK2;
        else counter <= counter - 1;
    end
endcase
end
end

```

4.5 Bidirectional Bus Control (Tri-State Buffers)

The I2C protocol requires an open-drain architecture, meaning devices can only pull the bus low (0) or release it (High-Z) to let pull-up resistors return it to a logical 1. Driving a 1 directly causes short circuits (bus contention). This physical constraint was implemented using a continuous assignment with a tri-state condition:

Verilog:

```

// Bidirectional SDA Line Control
assign i2c_sda = (write_enable == 1 && sda_out == 0) ? 1'b0 : 1'bz;

```

If the FSM has control of the bus (`write_enable == 1`) and wants to transmit a zero, it actively drives `1'b0`. In all other conditions—including when the FSM is reading data from a slave—it drives `1'bz` (high impedance), safely releasing the wire.

5. TESTING AND RESULTS

5.1 Simulation Setup and Environment

The verification of the I2C Master Controller was conducted using the Xilinx Vivado Design Suite logic simulator. A comprehensive Verilog testbench was developed to simulate a Host computer and a "Smart Slave" sensor device. To accurately reflect physical hardware conditions, the testbench simulated the open-drain architecture by utilizing Verilog pullup() primitives on both the SDA and SCL lines. The system clock was driven at 50 MHz, generating an internal I2C clock of 2.5 MHz

5.2 Test Case 1: Standard Write Operation

The first phase of testing verified the Master's ability to execute a standard I2C Write transaction without timing violations. The Host interface was triggered by asserting the enable signal high for 500ns. The target slave address was set to 0x50 (7'h50), the Read/Write (R/W) bit was set to 0 (Write), and the payload data was set to 0xAB (8'hAB).

As observed in the simulation waveform, the FSM successfully generates the START condition by pulling SDA low while SCL is high. It serially transmits the address, reads the acknowledgment from the slave, and subsequently shifts out the 10101011 (0xAB) data byte. The transaction cleanly terminates with a STOP condition, releasing the bus back to the IDLE state.

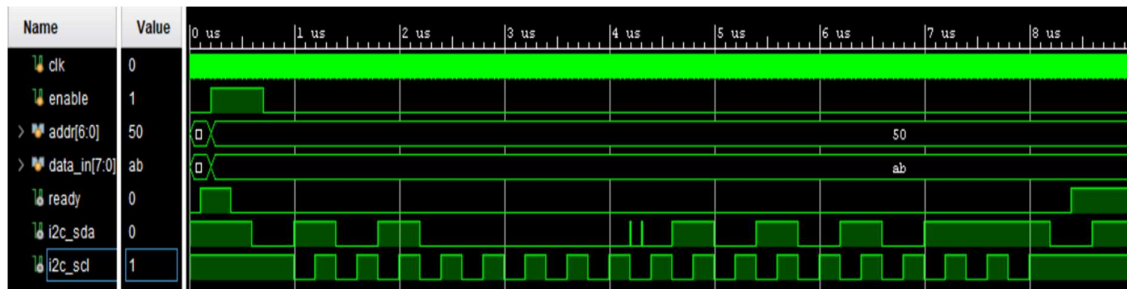


Figure 5.1: Simulation Waveform of a Successful I2C Write Operation (Address: 0x50, Data: 0xAB)

5.3 Test Case 2: Standard Read Operation

The second test case verified the Master's reception logic and open-drain bi-directional control. The Host triggered a Read transaction by setting the R/W bit to 1. The simulated "Smart Slave" was pre-loaded with a static payload byte of 0xC3 (8'hC3).

During the data phase, the Master FSM correctly disables its write_enable signal, releasing the SDA line to High-Impedance (Z). This allows the slave to pull the bus low to transmit the data. The Master's internal FSM captures the incoming serial bits on the negative edge of the clock. Once all 8 bits are received, the Master successfully concatenates them and updates the data_out register from XX (unknown) to the correct c3 hexadecimal value, proving bidirectional data integrity.

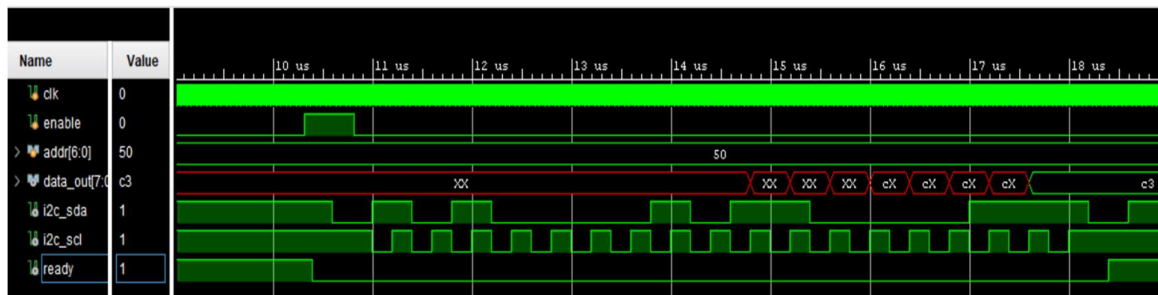


Figure 5.2: Simulation Waveform of an I2C Read Operation Capturing Slave Data (0xC3)

5.4 Test Case 3: Radiation Fault Injection and Mitigation

The final and most crucial test case validated the effectiveness of the Digital Temporal Voting Filter against Single Event Transients (SETs). During a simulated transaction, an artificial radiation strike was injected directly onto the physical SDA line exactly during the critical Slave Acknowledgment phase. Utilizing the Verilog force command, the line was violently driven high (1'b1) for exactly 15 nanoseconds, simulating a high-energy cosmic particle strike.

The simulation waveform perfectly illustrates the filter's success. The physical i2c_sda wire shows a sharp, erratic spike. The internal 3-bit shift register (sda_sr) instantly captures this corrupted data. However, because the transient pulse lasted less than the required 60-nanosecond verification window (3 clock cycles), the temporal voting algorithm rejected the anomaly. The filtered_sda signal, which the FSM relies

on, remained completely stable at 0. The Master Controller successfully ignored the radiation strike and completed the transaction without deadlocking.

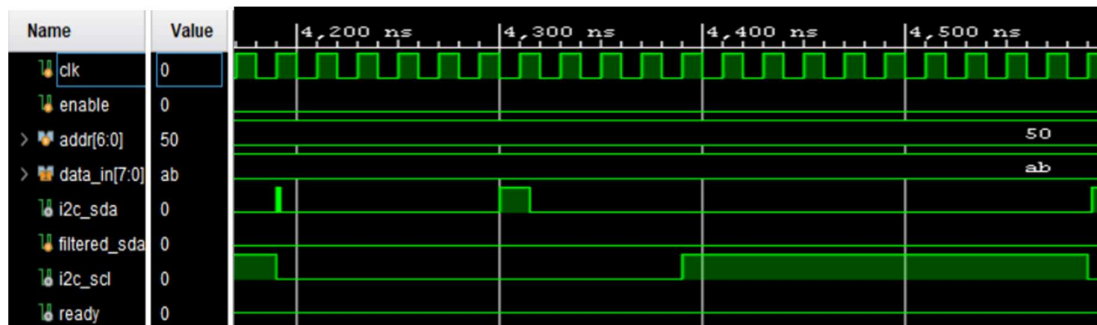


Figure 5.3: Fault Injection Simulation Demonstrating the Temporal Voting Filter Mitigating a 15ns Radiation Strike

5.5 Summary of Results

The simulation results confirm that the proposed hardware-driven FSM and digital filter architecture completely eliminate the delta-cycle race conditions and false START/STOP interpretations that plague standard software-based I2C controllers. The system guarantees deterministic data flow while autonomously absorbing physical-layer electrical disturbances, fulfilling the primary objective of creating a robust, space-grade communication interface.

6. CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

The fundamental fragility of the standard software-driven I2C protocol presents a significant risk to CubeSat telemetry systems operating in high-radiation environments. This project successfully designed, simulated, and verified a custom, radiation-hardened I2C Master Controller using Verilog HDL to directly address these vulnerabilities. By shifting the protocol management from software algorithms to a deterministic 9-state hardware Finite State Machine (FSM), the architecture completely eliminated delta-cycle race conditions and ensured strict physical bus arbitration.

Furthermore, the integration of a 3-stage Digital Temporal Voting Filter proved highly effective in mitigating Single Event Transients (SETs). Fault-injection simulations confirmed that the filter successfully identified and rejected a 15-nanosecond high-energy radiation spike on the data line, preventing the FSM from misinterpreting the noise as a false STOP condition or a missing acknowledgment. Operating at a stable 2.5 MHz internal clock speed, the controller executed flawless Read and Write transactions while autonomously absorbing physical-layer electrical disturbances. This hardware-first approach provides a robust, scalable, and highly reliable digital shield for space-grade nanosatellite communications, ensuring uninterrupted mission-critical data handling.

6.2 Future Scope

While the current architecture successfully mitigates transient voltage spikes (SETs), future iterations of this project can be expanded to handle permanent Single Event Upsets (SEUs) that cause the bus to deadlock. Proposed future work includes:

1. **Deadlock Recovery Sequence:** Implementing an autonomous 9-clock-cycle SCL generation sequence to force a stuck slave to release the SDA line if a transaction hangs indefinitely.
2. **Multi-Master Arbitration:** Expanding the FSM to support multi-master collision detection and bus arbitration, allowing multiple redundant onboard computers (OBCs) to share the same telemetry sensors.

REFERENCES

1. "I2C stuck bus: Prevention and workarounds," Texas Instruments, Dallas, TX, USA, Appl. Rep. SCPA069, Mar. 2003.
 2. A Basic Guide to I2C, Texas Instruments
 3. UM10204: I2C-bus specification and user manual, NXP Semiconductors, 7th Revision, October 2021
 4. J. Bouwmeester, M. Langer, and E. Gill, "Survey on the implementation and reliability of CubeSat electrical bus interfaces," CEAS Space Journal, vol. 9, no. 2
 5. R. A. Austin, N. Mahadevan, B. D. Sierawski, et al., "A CubeSat-payload radiation-reliability assurance case using goal structuring notation," in 2017 Annual Reliability and Maintainability Symposium (RAMS), 2017
 6. X. Cui and L. Liu, "Mitigating single event upset of FPGA for the onboard bus control of satellite," Microelectronics Reliability, vol. 114, p. 113831, 2020.
 7. J. Lofgren, "Circuit-level techniques for mitigating radiation-induced degradation of commercial microelectronics in space," Ph.D. dissertation, Stanford University, 2021.
 8. A. Albaloooshi, A. H. M. Jallad, and P. R. Marpu, "Fault analysis and mitigation techniques of the I2C bus for nanosatellite missions," IEEE Access, vol. 11, pp. 34709-34717, 2023.
 9. M. Suchitra, D. L. Girijamba, and B. Gokul, "Enhanced I2C Architecture With Integrated CRC for Error Detection in Multi-Slave Systems," in 2025 3rd International Conference on Smart Systems for applications in Electrical Sciences (ICSSES), 2025.
 10. A. Mishra and S. Rawat, "FPGA Improved I2C Protocol for Multiple Slave Device Using VHDL," International Journal of Microelectronics and Digital Integrated Circuits, 2025.\
 11. G. Ajitha, et al., "Ensuring I2C Bus Reliability in Nanosatellite Operations with Fault Analysis and Mitigation Strategies," International Journal of Scientific Research in Engineering and Management, 2025.
 12. P. Maillard, "Radiation Effects in FPGAs and SoCs," NSREC Short Course Archive, 2025.
-

APPENDIX

A. Abbreviations

- **I2C:** Inter-Integrated Circuit
- **SDA:** Serial Data Line
- **SCL:** Serial Clock Line
- **FSM:** Finite State Machine
- **SEU:** Single Event Upset
- **SET:** Single Event Transient
- **COTS:** Commercial Off-The-Shelf
- **HDL:** Hardware Description Language
- **RTL:** Register Transfer Level
- **OBC:** On-Board Computer

B. Complete Verilog Source Code: I2C Master Design

Verilog

```
`timescale 1ns / 1ps
```

```
module i2c_master(
```

```
    input wire clk,
```

```
    input wire [6:0] addr,
```

```
    input wire [7:0] data_in,
```

```
    input wire rst,
```

```
    input wire enable,
```

```
    input wire rw,
```

```
    output reg [7:0] data_out,
```

```
    output wire ready,
```

```
    inout i2c_scl,
```

```
    inout i2c_sda
```

```
);
```

```
localparam IDLE = 0;
```

```
localparam START = 1;
```

```
localparam ADDRESS = 2;
```

```
localparam READ_ACK = 3;
```

```

localparam WRITE_DATA = 4;
localparam WRITE_ACK = 5;
localparam READ_DATA = 6;
localparam READ_ACK2 = 7;
localparam STOP = 8;

localparam DIVIDE_BY = 20;

reg [7:0] state;
reg [7:0] saved_addr;
reg [7:0] saved_data;
reg [7:0] counter;
reg [7:0] counter2 = 0;
reg write_enable;
reg sda_out;
reg i2c_scl_enable = 0;
reg i2c_clk = 1;

reg [2:0] sda_sr = 3'b111;
reg filtered_sda = 1;

always @(posedge clk or posedge rst) begin
    if (rst == 1) begin
        sda_sr <= 3'b111;
        filtered_sda <= 1'b1;
    end else begin
        sda_sr <= {sda_sr[1:0], i2c_sda};
        if ({sda_sr[1:0], i2c_sda} == 3'b111) begin
            filtered_sda <= 1'b1;
        end else if ({sda_sr[1:0], i2c_sda} == 3'b000) begin
            filtered_sda <= 1'b0;
        end
    end
end
end

```

```

assign ready = ((rst==0) && (state == IDLE)) ? 1 : 0;
assign i2c_scl = (i2c_scl_enable == 0) ? 1 : i2c_clk;
assign i2c_sda = (write_enable == 1 && sda_out == 0) ? 1'b0 : 1'bz;

always @(negedge i2c_clk, posedge rst) begin
    if (rst == 1) begin
        write_enable <= 1;
        sda_out    <= 1;
    end else begin
        if ((state == READ_ACK) || (state == READ_DATA) || (state ==
WRITE_ACK)) begin
            write_enable <= 0;
        end else begin
            write_enable <= 1;
        end
    end

    if (state == START || state == STOP) begin
        sda_out <= 0;
    end else if (state == ADDRESS) begin
        sda_out <= saved_addr[counter];
    end else if (state == WRITE_DATA) begin
        sda_out <= saved_data[counter];
    end else begin
        sda_out <= 1;
    end
end
end

always @(posedge clk) begin
    if (counter2 == (DIVIDE_BY/2) - 1) begin
        i2c_clk <= ~i2c_clk;
        counter2 <= 0;
    end
end

```

```

        else counter2 <= counter2 + 1;
    end

    always @(negedge i2c_clk, posedge rst) begin
        if (rst == 1) begin
            i2c_scl_enable <= 0;
        end else begin
            if ((state == IDLE) || (state == START) || (state == STOP)) begin
                i2c_scl_enable <= 0;
            end else begin
                i2c_scl_enable <= 1;
            end
        end
    end
end

```

```

always @(negedge i2c_clk, posedge rst) begin
    if(rst == 1) begin
        state <= IDLE;
    end
    else begin
        case(state)
            IDLE: begin
                if (enable) begin
                    state <= START;
                    saved_addr <= {addr, rw};
                    saved_data <= data_in;
                end
                else state <= IDLE;
            end
            START: begin
                counter <= 7;
                state <= ADDRESS;
            end
            ADDRESS: begin

```

```

        if (counter == 0) begin
            state <= READ_ACK;
        end else counter <= counter - 1;
    end
    READ_ACK: begin
        if (filtered_sda == 0) begin
            counter <= 7;
            if(saved_addr[0] == 0) state <= WRITE_DATA;
            else state <= READ_DATA;
        end else begin
            state <= STOP;
        end
    end
    WRITE_DATA: begin
        if (counter == 0) begin
            state <= WRITE_ACK;
        end else counter <= counter - 1;
    end
    WRITE_ACK: begin
        if (filtered_sda == 0) state <= STOP;
        else state <= STOP;
    end
    READ_DATA: begin
        data_out[counter] <= filtered_sda;
        if (counter == 0) state <= READ_ACK2;
        else counter <= counter - 1;
    end
    READ_ACK2: begin
        state <= STOP;
    end
    STOP: begin
        state <= IDLE;
    end
endcase

```

```
    end
end
endmodule
```

C. Complete Verilog Source Code: I2C Testbench

Verilog:

```
`timescale 1ns / 1ps
module tb_i2c_master;
    reg clk;
    reg rst;
    reg enable;
    reg rw;
    reg [6:0] addr;
    reg [7:0] data_in;
    wire [7:0] data_out;
    wire ready;
    wire i2c_sda;
    wire i2c_scl;
    reg tb_sda_drive;

    assign i2c_sda = (tb_sda_drive == 1'b0) ? 1'b0 : 1'b1;

    i2c_master uut (
        .clk(clk),
        .rst(rst),
        .addr(addr),
        .data_in(data_in),
        .enable(enable),
        .rw(rw),
        .data_out(data_out),
        .ready(ready),
        .i2c_scl(i2c_scl),
        .i2c_sda(i2c_sda)
    );
endmodule
```

```
);
```

```
initial begin
```

```
    clk = 0;
```

```
    forever #10 clk = ~clk;
```

```
end
```

```
pullup(i2c_sda);
```

```
pullup(i2c_scl);
```

```
initial begin
```

```
    rst = 1;
```

```
    enable = 0;
```

```
    rw = 0;
```

```
    addr = 0;
```

```
    data_in = 0;
```

```
    tb_sda_drive = 1'bz;
```

```
    #100;
```

```
    rst = 0;
```

```
    #100;
```

```
    $display("Starting Write Transaction...");
```

```
    addr = 7'h50;
```

```
    data_in = 8'hAB;
```

```
    rw = 0;
```

```
    enable = 1;
```

```
    #500 enable = 0;
```

```
    #3600;
```

```
    $display("Radiation Strike! Forcing SDA High.");
```

```
    force i2c_sda = 1'b1;
```

```
    #15;
```

```
    release i2c_sda;
```



```

$display("Radiation Passed. Bus restored.");

#8000;

$display("Starting Read Transaction...");
addr = 7'h50;
rw = 1;
enable = 1;
#500 enable = 0;
end

integer bit_count = 0;
reg [7:0] slave_tx_data = 8'hC3;
reg in_transaction = 0;

always @(negedge i2c_sda) begin
    #1;
    if (i2c_scl === 1'b1 && in_transaction == 0) begin
        in_transaction = 1;
        bit_count = 0;
    end
end

always @(posedge i2c_sda) begin
    #1;
    if (i2c_scl === 1'b1) begin
        in_transaction = 0;
    end
end

always @(negedge i2c_scl) begin
    #2;
    if (in_transaction) begin
        bit_count = bit_count + 1;
    end
end

```

```
    if (bit_count <= 8) begin
        tb_sda_drive <= 1'bz;
    end else if (bit_count == 9) begin
        tb_sda_drive <= 1'b0;
    end else if (bit_count >= 10 && bit_count <= 17) begin
        tb_sda_drive <= (rw == 1) ? slave_tx_data[17 - bit_count] : 1'bz;
    end else if (bit_count == 18) begin
        tb_sda_drive <= 1'bz;
        bit_count = 9;
    end
end
end
end
endmodule
```